

Quantum CNNs, Transformers, and Structured Architectures

EVA: Quantum Machine Learning – Lecture 10

Pavel Sulimov

Spring 2026

ZHAW Zürcher Hochschule für Angewandte Wissenschaften

1. Inductive biases and structured quantum architectures
2. Quadvolutional Neural Networks (quadvolution)
3. Quantum Convolutional Neural Networks (QCNN)
4. Quantum attention and transformers
5. Shallow circuits on real hardware: time-series experiments
6. Practical constraints and when structure helps

Part 1. Inductive biases and structured architectures

Why structure matters in classical deep learning

Classical deep learning owes much of its practical success to inductive biases encoded in the architecture:

Architecture	Inductive bias	Useful when
Convolutional NN (CNN)	locality, translation equivariance	images, signals
Recurrent NN (RNN/LSTM)	sequential order, memory	time series, text
Transformer	global attention, permutation equivariance	sequences, sets
Graph NN (GNN)	local neighborhood structure	graphs, molecules

A fully connected network can learn all of these in principle, but it requires more data and more parameters to discover the structure that is already obvious from the problem.

Inductive biases in quantum circuits

The same principle applies to quantum circuits. A fully parameterized circuit (hardware-efficient ansatz) has no built-in structure and will generically suffer from barren plateaus as qubit count grows.

Structured architectures impose a prior on the computation:

Architecture	Imposed structure
QCNN	locality (neighboring qubits), pooling hierarchy
Quanvolution	local receptive fields (sliding window over patches)
Equivariant QNN	symmetry group of the problem (e.g. permutation)
Quantum reservoir	fixed dynamics, only readout is trained
Quantum attention	doubly stochastic attention from unitary circuits

The payoff: fewer parameters, slower growth of barren plateaus, and a prior that matches the task structure.

Part 2. Quanyvolutional Neural Networks

Quanvolution: the core idea

Henderson et al. (2020) proposed replacing the first convolutional layer of a classical CNN with a *quanvolutional* layer.

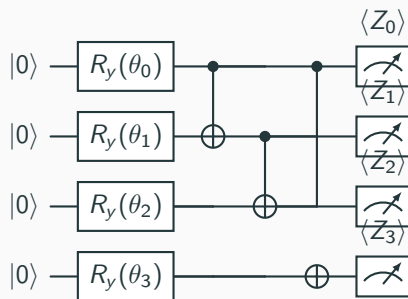
The procedure mirrors classical convolution:

1. Slide a small window (e.g. 2×2) over the input image.
2. For each patch, encode the k pixel values as rotation angles on k qubits.
3. Apply a fixed (untrained) random quantum circuit to those qubits.
4. Measure expectation values $\langle Z_i \rangle$ to produce k output channels.
5. Stack these feature maps and pass them into a classical convolutional or fully connected head.

The quantum circuit acts as a **non-linear random feature map** applied locally to each image patch.

Quanvolution: circuit for one patch

For a 2×2 patch with pixel values $p_0, p_1, p_2, p_3 \in [0, 1]$, scaled to angles $\theta_i = \pi p_i$:



The circuit is the same for every patch (weight sharing in the quantum sense). The output is 4 real numbers forming one spatial location in the feature map. Sliding over a 28×28 image with stride 2 produces $14 \times 14 \times 4$ features.

Quanvolution: what is learned vs fixed?

In the original Henderson et al. formulation, the quantum circuit is **fixed and random** – it is not trained.

This is analogous to random kitchen sinks or random Fourier features in classical ML: a fixed non-linear random map followed by a trained linear readout can already achieve competitive performance on simple tasks.

Variants in later literature:

- **Trained quanvolution:** make the circuit parameters trainable. This is closer to a standard convolutional layer but uses quantum gates.
- **Ensemble:** use multiple different random circuits to produce more output channels (increases feature diversity at the cost of circuit calls).
- **Hybrid:** freeze the quantum layer and fine-tune only the classical head (transfer learning analogy).

Quanvolution: cost and practical limits

The computational cost of quanvolution scales as:

$$\text{cost} = N_{\text{patches}} \times S \times C_{\text{circuit}},$$

where N_{patches} is the number of image patches, S is the number of shots, and C_{circuit} is the cost per circuit execution.

For a 28×28 MNIST image with 2×2 patches and stride 1: $N_{\text{patches}} = 27 \times 27 = 729$ circuits per image. With 1000-shot sampling this is already slow on classical simulators.

Practical recommendations:

- use a statevector simulator (exact, no shot noise) for training studies
- use stride > 1 to reduce patch count
- restrict quanvolution to the first layer; use classical CNN for the rest
- on real hardware, the circuit depth must stay within the device coherence time

Part 3. Quantum Convolutional Neural Networks

QCNN: architecture overview

Cong, Choi, and Lukin (Nature Physics, 2019) introduced the Quantum Convolutional Neural Network (QCNN) for classifying quantum states directly.

The architecture mirrors classical CNNs:

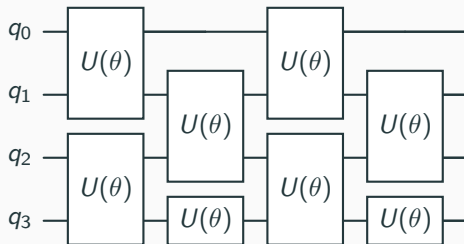


Convolution layer: apply a 2-qubit parameterized unitary to all neighboring qubit pairs (and next-nearest neighbors), then repeat.

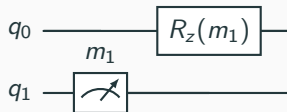
Pooling layer: measure a subset of qubits; apply a conditional single-qubit rotation on the remaining qubits based on the measurement outcome. This halves the number of active qubits.

QCNN: convolution and pooling circuits

Convolution layer (even-odd pairs on 8 qubits, one layer shown):



Pooling layer (odd qubits measured, even qubits updated):



QCNN: why it avoids barren plateaus

Standard random ansatz circuits exhibit gradient variance $\text{Var}[\partial\mathcal{L}/\partial\theta] = \mathcal{O}(2^{-n})$, causing barren plateaus.

QCNN avoids this through two properties:

- **Local cost function:** each pooling layer measures a local observable (a single qubit), not a global one. Local observables have gradients that scale polynomially rather than exponentially.
- **Hierarchical structure:** the circuit depth grows as $\mathcal{O}(\log n)$ rather than $\mathcal{O}(n)$, limiting the effective barren-plateau regime.

Cong et al. proved that with a local cost and logarithmic depth, the gradient variance is $\Omega(1/\text{poly}(n))$ – exponentially better than global cost circuits of the same qubit count.

Caveat: this analysis holds for specific QCNN constructions and may not apply to arbitrary modifications of the architecture.

QCNN: application and limitations

The original QCNN was designed for **quantum state classification** – the input is already a quantum state on n qubits, not classical data.

Applications in the original paper:

- classify 1D symmetry-protected topological phases from their ground states
- detect quantum phase transitions in condensed matter systems

Extensions to classical data:

- encode classical image patches with angle encoding, then apply QCNN layers
- hybrid: use classical CNN to extract patches, encode as quantum states, apply QCNN for feature compression

Current limitation: for classical data classification tasks on benchmark datasets, classical CNNs outperform QCNN by a large margin due to the limited number of qubits. The QCNN

Part 4. Quantum attention and transformers

Classical self-attention: recap

The classical transformer's attention mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V.$$

Key properties of the attention matrix $A = \text{softmax}(QK^\top / \sqrt{d})$:

- each row sums to 1 (row-stochastic): $\sum_j A_{ij} = 1$
- all entries are non-negative: $A_{ij} \geq 0$
- the rows represent attention distributions over tokens

This is a *singly stochastic* matrix. The softmax enforces row normalization but places no constraint on columns – multiple tokens can all attend strongly to the same target.

Doubly stochastic matrices from unitary circuits

A matrix B is **doubly stochastic** if:

$$B_{ij} \geq 0, \quad \sum_j B_{ij} = 1 \text{ (rows)}, \quad \sum_i B_{ij} = 1 \text{ (columns)}.$$

Key fact: for any unitary matrix U , the matrix of squared magnitudes $B_{ij} = |U_{ij}|^2$ is doubly stochastic.

Proof: rows: $\sum_j |U_{ij}|^2 = 1$ (unit norm of each row). Columns: $\sum_i |U_{ij}|^2 = 1$ (unit norm of each column since $U^\dagger U = I$).

This gives a route to doubly stochastic attention weights from a parameterized quantum circuit: let $U(\theta)$ be a unitary on n qubits, and define $A_{ij} = |U_{ij}(\theta)|^2$.

Doubly stochastic attention ensures that no single key position dominates all query positions simultaneously – a structural constraint absent from softmax attention.

Quantum Doubly Stochastic Transformer

Born et al. (NeurIPS 2025) proposed the **Quantum Doubly Stochastic Transformer (QDST)**: a transformer variant where the attention matrix is derived from a parameterized quantum circuit.

Construction:

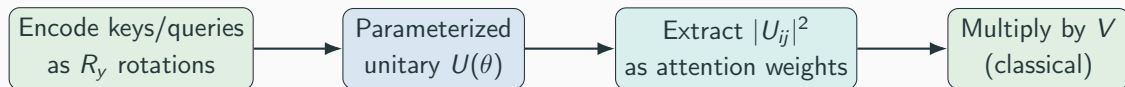
1. Encode key and query vectors as rotation angles into a quantum circuit.
2. Apply a sequence of parameterized 2-qubit gates.
3. Extract $A_{ij} = |U_{ij}(\theta)|^2$ as the attention weights.
4. Multiply by the value matrix V in the classical head.

Reported advantages:

- column normalization prevents attention collapse (all attention on one token)
- the unitary structure acts as an implicit regularizer
- competitive performance on small sequence tasks at matched parameter count

Quantum attention: practical construction

For a sequence of n tokens, a minimal quantum attention layer:



Attention matrix entry $A_{ij} = |\langle j | U(\theta) | i \rangle|^2$ measures how much the state $|i\rangle$ (query i) overlaps with output basis state $|j\rangle$ (key j) after the unitary evolution.

lecture_10_demo.ipynb, Demo 2: minimal 4-token quantum attention.

Part 5. Shallow circuits on real hardware

Shallow entangled circuits for time series

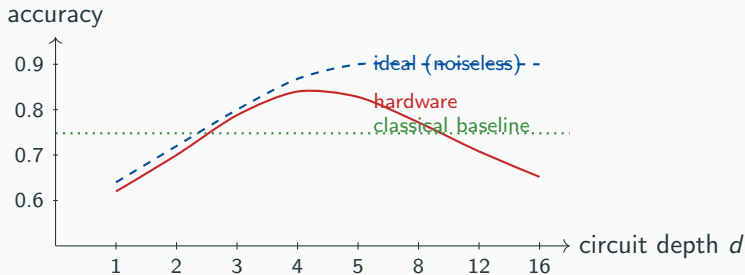
A 2025 IBM study (Nature Scientific Reports) evaluated shallow parameterized circuits on real superconducting hardware for time-series prediction.

Key findings:

- Circuits with depth 2–4 (within hardware coherence limits) can capture temporal correlations in short financial and sensor time series.
- The entangling structure (which qubits are connected) matters more than the choice of single-qubit rotation gates.
- Noise has a non-trivial effect: at very low depth, quantum noise acts as a form of regularization; at high depth it dominates and degrades performance.
- Classical baselines (ARIMA, linear regression) still match or exceed these circuits on standard benchmarks, but the gap narrows for tasks with genuinely non-classical correlation structure.

Circuit depth vs accuracy on NISQ devices

For a time-series prediction task with a 5-qubit circuit on IBM hardware:



The optimal hardware depth is a hardware-specific sweet spot: too shallow to learn, too deep to survive noise.

Implications for architecture design on NISQ hardware

Three practical guidelines from the shallow-circuit experiments:

- 1. Match circuit depth to coherence time.** On current IBM superconducting devices, depths of 2–6 two-qubit gate layers are reliable. For longer computations, error mitigation (ZNE, twirling) is required.
- 2. Prefer local over global entanglement.** Nearest-neighbor entangling gates have lower error rates and shorter gate times than long-range operations. This aligns naturally with convolutional structure.
- 3. Exploit the quantum noise.** Low-level depolarizing noise can act as implicit regularization (a “quantum dropout”) at short circuit depths. At larger depths the same noise becomes destructive. Deliberately calibrated noise injection has been explored as a feature in quantum reservoir computing (Lecture 11).

Part 6. Practical constraints

When does structure help in QML?

Task type	Recommended architecture	Reason
Image classification (classical data)	Quangvolution + classical CNN	Local receptive fields match spatial structure
Quantum state classification	QCNN	Designed for quantum-native input; avoids barren plateaus
Sequence / set modeling	Quantum attention (QDST)	Doubly stochastic prior prevents attention collapse
Time series on NISQ hardware	Shallow circuits (depth ≤ 4)	Hardware noise limits usable depth
General regression/classification	HEA or re-uploading (Lecture 9)	No strong structural prior available

Comparison: structured vs unstructured

Property	HEA (unstructured)	QCNN	Quanvolution
Circuit depth	$O(n \cdot L)$	$O(\log n)$	shallow (1–2 layers)
Barren plateau risk	high	low	N/A (not trained)
Gradient scaling	$O(2^{-n})$	$\Omega(1/\text{poly}(n))$	no gradient
Hardware depth	deep	moderate	shallow
Data type	any	quantum states	image patches
Training needed	yes	yes	no (random)

Summary

Topic	Key takeaway
Inductive bias	Structure reduces parameters, eases optimization
Quanvolution	Fixed random circuits extract local features from image patches
QCNN	Logarithmic depth, local cost functions avoid barren plateaus
Doubly stochastic attention	$ U_{ij} ^2$ from unitaries enforces column normalization
QDST	Quantum circuit produces attention weights; classical simulation so far
Shallow NISQ circuits	Depth 2–6 is the practical window on current hardware
When to use	Match architecture to data structure and qubit count

Lecture 11 moves to **quantum reservoir computing**: fixed quantum dynamics with only the readout layer trained, which sidesteps the training problem entirely.